Software Testing and Validation

Second Project — Report

Dinis Rodrigues (ist106213) Artur Krystopchuk (ist104145)
Gabriel Santos (ist1116334)

June 2026

Resumo

This report presents the design of test cases for the urban delivery system described in the specification of the second project for the Software Testing and Validation course. Appropriate design strategies are applied to test the Order and Delivery classes at the class level, as well as the `computeDeliveryCost()` and `updateItem()` methods. The final section presents the TestNG implementation of eight selected test cases from the test suite of the Order class.

Índice

1 Class-Level Testing of the Order Class **2**
1.1 Chosen Strategy: Non-Modal Class Testing . . . 2
1.2 On/Off Points . . . 2
1.3 Domain Matrix with Test Vectors . . . 2
1.4 Relational Invariant (Same Supplier) . . . 4

2 Class-Level Testing of the Delivery Class **4**
2.1 State Machine . . . 4
2.2 Valid and Invalid Transitions . . . 4
2.3 Strategy: *Round-Trip / Transition Coverage* . . . 5
2.4 Test Cases . . . 5

3 Method `computeDeliveryCost()` **5**
3.1 Strategy: Decision Table / Boundary Analysis . . . 5
3.2 Decision Table . . . 5
3.3 Test Cases . . . 5

4 Method `updateItem()` **5**
4.1 Chosen Strategy . . . 5
4.2 State + Input Combinations . . . 5
4.3 Test Cases . . . 5

5 Implementation in TestNG **5**

1 Class-Level Testing of the **Order** Class

1.1 Chosen Strategy: Non-Modal Class Testing

The Order class does not possess a state machine (there are no modes that alter method behaviour); its class-level behaviour is determined by a set of *invariants* over its instance variables. The appropriate strategy for testing a class of this type is **Non-Modal Class Testing**, which consists of: (i) identifying the domains of the variables; (ii) determining the *On*/*Off* points for each invariant boundary; and (iii) constructing a *domain matrix* with test vectors, where each test case exercises one boundary at a time, keeping the remaining variables at typical values.

1.2 On/Off Points

Boundary (invariant)	**On**	**Off**
client $\neq$ `null`	Valid client	`null`
addr $\neq$ `null`	Valid string	`null`
addrLength ≥ 1	1	0 (empty)
distance ≥ 0	0	-1
distance ≤ 20000	20000	20001
quantity ≥ 1	1	0
quantity ≤ 10	10	11
cost ≤ 1000	1000	1000.01
totalQty $\leq 20 + \lfloor \text{cost}/50 \rfloor$	= limit	limit + 1

1.3 Domain Matrix with Test Vectors

Each column represents a test case. In each column, exactly one boundary is in *On* or *Off* (shaded cell), while the remaining variables are at a typical value (row *In*). Adopted typical value: order with one item of price 50 and quantity 4 (cost = 200, totalQty = 4, limit = 24), address length of 3, and distance = 5000.

Tabela 2: Domain matrix for the Order class.

Var.	Condition		1	2	3	4	5	6	7	8	9	10
addr	≥ 1	On	1									
		Off		0								
	Typical	In			3	3	3	3	3	3	3	3
distance	≥ 0	On			0							
		Off				−1						
	≤ 20000	On					20000					
		Off						20001				
	Typical	In	5000	5000					5000	5000	5000	5000
quantity	≥ 1	On							1			
		Off								0		
	≤ 10	On									10	
		Off										11
	Typical	In	4	4	4	4	4	4				
cost	≤ 1000	On										
		Off										
	Typical	In	200	200	200	200	200	200	50	0	500	550
totalQty	$\leq$ lim.	On										
		Off										
	Typical	In	4	4	4	4	4	4	1	0	10	11
Expected result			✓	✗	✓	✗	✓	✗	✓	✗	✓	✗

Tabela 3: Domain matrix for the Order class (continuation).

Var.	Condition		11	12	13	14	15	16	17	18
client	$\neq$ `null`	On					ok			
		Off						`null`		
	Typical	In	ok	ok	ok	ok			ok	ok
addr	$\neq$ `null`	On							"x"	
		Off								`null`
	typical length	In	3	3	3	3	3	3		
distance	Typical	In	5000	5000	5000	5000	5000	5000	5000	5000
quantity	Typical	In	—	—	—	—	—	—	—	—
cost	≤ 1000	On	1000							
		Off		1000.01						
	Typical	In			200	200	200	200	200	200
totalQty	$\leq$ lim.	On			24					
		Off				25				
	Typical	In	20	20			4	4	4	4
Expected result			✓	✗	✓	✗	✓	✗	✓	✗

Legend: ✓ = valid state (operation accepted); ✗ = invalid state (must throw `InvalidOperatio`

`nException` without changing the order state, according to I6).

1.4 Relational Invariant (Same Supplier)

This invariant is non-numeric, and therefore does not have On/Off points. It is tested with two dedicated cases:

#	Scenario	Expected Result
19	Add item from supplier S to an order already containing items from supplier S	Accepted (✓)
20	Add item from supplier S' to an order already containing items from supplier S	`InvalidOperationException` (✗)

2 Class-Level Testing of the **Delivery** Class

2.1 State Machine

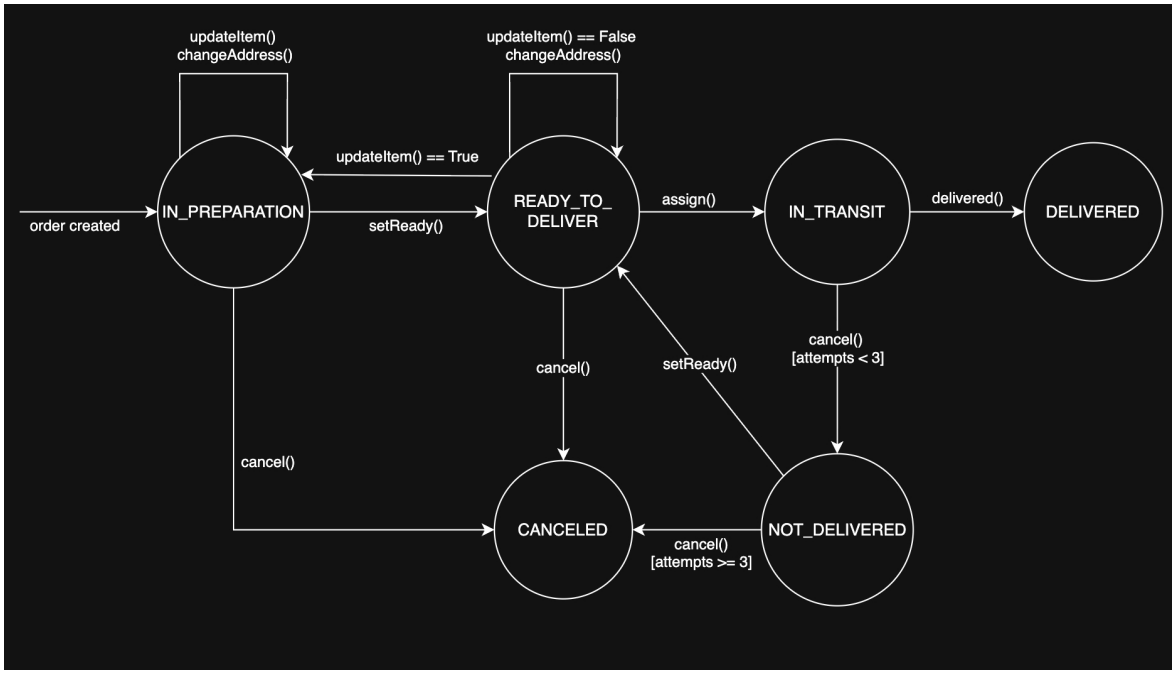


Figura 1: State machine of the Delivery class.

2.2 Valid and Invalid Transitions

Transition table to be filled.

2.3 Strategy: *Round-Trip / Transition Coverage*

2.4 Test Cases

3 Method `computeDeliveryCost()`

3.1 Strategy: Decision Table / Boundary Analysis

3.2 Decision Table

3.3 Test Cases

4 Method `updateItem()`

4.1 Chosen Strategy

4.2 State + Input Combinations

4.3 Test Cases

5 Implementation in TestNG

Four successful cases and four unsuccessful cases were selected from the test suite of the Order class (Section 1).

```
package supereats.tests;

import org.testng.annotations.*;
import static org.testng.Assert.*;

import supereats.core.*;

public class OrderTest {

  private static final double EPS = 1e-9;
  private Supplier s1;       // reference supplier
  private Supplier s2;       // distinct supplier
  private Client   client;
  // Items from supplier s1 (weight in grams, integer price)
  private Item i1;           // price 10
  private Item i2;           // price  5
  private Item i50;          // price 50 (for totalQty cases)

  // Item from a distinct supplier (s2)
  private Item iOther;

  @BeforeMethod
  public void setUp() {
    s1     = new Supplier("Loja A");
    s2     = new Supplier("Loja B");
    client = newClient();

    i1     = new Item(100,  10, s1, "Item 1");
    i2     = new Item(200,   5, s1, "Item 2");
    i50    = new Item(150,  50, s1, "Item 50");
    iOther = new Item(100,  10, s2, "Item from another supplier");
  }

  % Helper: the existence of a Client constructor is assumed.
  private Client newClient() {
    return new Client(/* ... valid data ... */);
  }
```

```
38
39  // ------------------------------------------------------------------------
40  // SUCCESS Cases
41  // ------------------------------------------------------------------------
42
43  // TC1: valid construction - all invariants satisfied.
44  @Test
45  public void tc1_validConstruction() {
46    Order o = new Order(client, "Street A", 5000);
47
48    assertEquals(o.totalQuantity(), 0);
49    assertEquals(o.cost(), 0.0, EPS);
50    assertTrue(o.itens().isEmpty());
51  }
52
53  // TC9: addItem in an empty order with a valid quantity.
54  @Test
55  public void TC9AddItemEmptyOrderValidQuantity() {
56    Order o = new Order(client, "Street A", 5000);
57
58    o.addItem(i1, 1);
59    assertTrue(o.contains(i1));
60    assertEquals(o.quantity(i1), 1);
61    assertEquals(o.cost(), 10.0, EPS);
62  }
63
64  // TC13: boundary totalQty <= 20 + floor(cost/50) (On).
65  // A unit price of 1 is used to isolate the totalQty boundary without
66  // violating the cost limit (I3).
67  // With cost = 24 * 1 = 24, the limit is
68  // 20 + floor(24/50) = 20 + 0 = 20... therefore it is necessary to raise the
69  // cost enough so that the limit reaches 24:
70  //   limit = 24  <=>  floor(cost/50) = 4  <=>  cost in [200, 250).
71  // Cost = 240 is chosen with items of price 10 (24 units * 10 = 240),
72  // keeping cost <= 1000.
73  @Test
74  public void TC13TotalQuantityAtDynamicLimit() {
75    Order o = new Order(client, "Street A", 5000);
76    // 3 items of price 10, quantities 10 + 10 + 4 = 24 units
77    Item a = new Item(150, 10, s1, "A");
78    Item b = new Item(150, 10, s1, "B");
79    Item c = new Item(150, 10, s1, "C");
80
81    o.addItem(a, 10);
82    o.addItem(b, 10);
83    o.addItem(c,  4);
84
85    assertEquals(o.totalQuantity(), 24); // = 20 + floor (240/50) = 24
86    assertEquals(o.cost(), 240.0, EPS);
87  }
88
89  // TC19: adding an item from the SAME supplier is accepted.
90  @Test
91  public void TC19SameSupplierIsAccepted() {
92    Order o = new Order(client, "Street A", 5000);
93
94    o.addItem(i1, 2);
95    o.addItem(i2, 3); // i2 also from supplier s1
96
97    assertTrue(o.contains(i1));
98    assertTrue(o.contains(i2));
99    assertEquals(o.totalQuantity(), 5);
100   assertEquals(o.cost(), 2 * 10 + 3 * 5, EPS);
```

```
101   }
102
103   // ------------------------------------------------------------------------
104   // FAILURE Cases
105   // ------------------------------------------------------------------------
106
107   // TC2: boundary addrLength >= 1 (Off) - empty address.
108   @Test(expectedExceptions = InvalidOperationException.class)
109   public void TC2EmptyAddressThrows() {
110     new Order(client, "", 5000);
111   }
112
113   // TC6: null client.
114   @Test(expectedExceptions = InvalidOperationException.class)
115   public void TC6NullClientThrows() {
116     new Order(null, "Street A", 5000);
117   }
118
119   // TC10: boundary quantity <= 10 (Off) - quantity 11.
120   @Test
121   public void TC10QuantityAboveMaxThrowsAndKeepsState() {
122     Order o = new Order(client, "Street A", 5000);
123     try {
124       o.addItem(i1, 11);
125       fail("Expected InvalidOperationException");
126     } catch (InvalidOperationException e) {
127       assertFalse(o.contains(i1)); // state unchanged
128       assertEquals(o.totalQuantity(), 0);
129     }
130   }
131
132   // TC20: adding an item from a DISTINCT supplier.
133   @Test(expectedExceptions = InvalidOperationException.class)
134   public void TC20DifferentSupplierThrows() {
135     Order o = new Order(client, "Street A", 5000);
136     o.addItem(i1,      1); // supplier s1
137     o.addItem(iOther, 1); // supplier s2 -> exception
138   }
139 }
```

Listing 1: Skeleton of the TestNG test class for Order.